

Laboratory Report: Lab 2

Zofia Warzycha, 280971

April 2026

1 Task 1: Sandwich Package

1.1 Task Description

For the first task, I implemented a program to create and calculate the calories of sandwiches. I used encapsulation to hide the class fields (making them `private`) and used getters to access them. I created a base `Ingredient` class and specific subclasses (`Meat`, `Veggies`, `Cheese`, `Sauce`) with different calorie modifiers.

I also created static factory methods (like `create()`) to validate the data before the object is made. If the weight or name is incorrect, the method prints an error and returns `null`. Finally, I put everything together in the `Sandwich` class, which stores ingredients in a `List` and calculates the total calories.

1.2 Crucial Part of the Code

The most important part of this task was data validation, encapsulation, and calorie calculation. Here is my complete `Sandwich` class implementation showing how ingredients are safely added and how total calories are calculated.

Listing 1: Sandwich class implementation

```
1 package SandwichPackage;
2
3 import java.util.ArrayList;
4 import java.util.Collections;
5 import java.util.List;
6
7 public class Sandwich {
8     private String name;
9     private Bread bread;
10    private List<Ingredient> ingredients;
11
12    private Sandwich(String name, Bread bread) {
13        this.name = name;
14        this.bread = bread;
15        this.ingredients = new ArrayList<>();
16    }
17
18    public static Sandwich create(String name, Bread bread) {
19        if (name == null || name.trim().isEmpty() || name.trim().length
            () > 15) {
```

```

20         System.out.println("Sandwich name should be between 1 and
21         15 symbols.");
22         return null;
23     }
24     if (bread == null) {
25         return null;
26     }
27     return new Sandwich(name.trim(), bread);
28 }
29 public boolean addIngredient(Ingredient ingredient) {
30     if (ingredient == null) {
31         return false;
32     }
33     if (ingredients.size() >= 10) {
34         System.out.println("The number of ingredients should be in
35         the range [0..10].");
36         return false;
37     }
38     ingredients.add(ingredient);
39     return true;
40 }
41 public double calculateTotalCalories() {
42     double totalCalories = bread.calculateCalories();
43     for (Ingredient ingredient : ingredients) {
44         totalCalories += ingredient.calculateCalories();
45     }
46     return totalCalories;
47 }
48 public List<Ingredient> getIngredients() {
49     return Collections.unmodifiableList(ingredients);
50 }
51 @Override
52 public String toString() {
53     String result = "Sandwich name: " + name + System.lineSeparator
54     ();
55     result += "Bread: " + bread.toString() + System.lineSeparator()
56     ;
57     result += "Ingredients: " + System.lineSeparator();
58
59     if (ingredients.isEmpty()) {
60         result += "- no ingredients" + System.lineSeparator();
61     } else {
62         for (Ingredient ingredient : ingredients) {
63             result += "- " + ingredient.toString() + System.
64             lineSeparator();
65         }
66     }
67     result += String.format("Calories: %.2f",
68         calculateTotalCalories()) + System.lineSeparator();
69     return result;
70 }

```

1.3 Tester Class

I wrote a `SandwichTester` class to check if my validation and calculations work correctly. It tests a healthy sandwich, invalid inputs, and the 10-ingredient limit.

Listing 2: Program tester

```
1 package SandwichPackage;
2
3 public class SandwichTester {
4     public static void main(String[] args) {
5         System.out.println("=== Creating a valid sandwich ===");
6         Bread validBread = Bread.create("wholegrain", "toasted", 100);
7         Sandwich healthySub = Sandwich.create("HealthySub", validBread
8             );
9
10        if (healthySub != null) {
11            healthySub.addIngredient(Meat.create(30));
12            healthySub.addIngredient(Cheese.create(20));
13            healthySub.addIngredient(Veggies.create(15));
14            healthySub.addIngredient(Sauce.create(10));
15            System.out.println(healthySub);
16        }
17
18        System.out.println("\n=== Invalid examples ===");
19        Bread badType = Bread.create("rye", "fresh", 100);
20        Bread badPrep = Bread.create("white", "burnt", 100);
21        Bread badWeight = Bread.create("white", "fresh", 300);
22        Ingredient badMeat = Meat.create(100);
23        Sandwich badName = Sandwich.create("
24            ThisNameIsWayTooLongForThatProgram", validBread);
25
26        System.out.println("\n=== Limit of ingredients test ===");
27        Sandwich loadedSub = Sandwich.create("LoadedSub", Bread.create
28            ("white", "fresh", 150));
29        if (loadedSub != null) {
30            for (int i = 1; i <= 11; i++) {
31                boolean added = loadedSub.addIngredient(Cheese.create
32                    (5));
33                System.out.println("Adding ingredient " + i + ": " +
34                    added);
35            }
36            System.out.println();
37            System.out.println(loadedSub);
38        }
39    }
40 }
```

1.4 Console Output

Example output produced by the program:

```
=== Creating a valid sandwich ===
Sandwich name: Healthy Sub
Bread: toasted wholegrain bread (100.0 g)
Ingredients:
- Meat (30.0g)
- Cheese (20.0g)
```

```
- Veggies (15.0g)
- Sauce (10.0g)
Calories: 383,00
```

```
=== Invalid examples ===
Invalid type of bread.
Invalid bread preparation.
Bread weight should be in the range [1..200].
Meat weight should be in the range [1..50].
Sandwich name should be between 1 and 15 symbols.
```

```
=== Limit of ingredients test ===
Adding ingredient 1: true
Adding ingredient 2: true
Adding ingredient 3: true
Adding ingredient 4: true
Adding ingredient 5: true
Adding ingredient 6: true
Adding ingredient 7: true
Adding ingredient 8: true
Adding ingredient 9: true
Adding ingredient 10: true
The number of ingredients should be in the range [0..10].
Adding ingredient 11: false
```

```
Sandwich name: Loaded Sub
Bread: fresh white bread (150.0 g)
Ingredients:
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
- Cheese (5.0g)
Calories: 540,00
```

2 Task 2: Space Station Management

2.1 Task Description

In the second part of the lab, I continued expanding the Space Station project. First, I updated the `CrewMember` class by adding a `UUID` `id` field, so every member now has a unique identifier. Then, I created three entirely new classes: `Officer`, `Supply`, and `Module`. Finally, I designed the main `SpaceStation` class to store lists of officers, supplies, and an array of crew members.

2.2 Crucial Part of the Code

For the new data classes, I made sure to use proper encapsulation by keeping fields `private` and exposing data only through getters. Here is the code for the `Officer` class as an example (the `Supply` class was implemented analogously):

Listing 3: Officer class implementation

```
1 import java.util.UUID;
2
3 public class Officer {
4     private UUID id;
5     private String name;
6     private String rank;
7     private String department;
8     private int yearsOfExperience;
9
10    public Officer(String name, String rank, String department, int
        yearsOfExperience) {
11        this.id = UUID.randomUUID();
12        this.name = name;
13        this.rank = rank;
14        this.department = department;
15        this.yearsOfExperience = yearsOfExperience;
16    }
17
18    public UUID getId() { return this.id; }
19    public String getName() { return this.name; }
20 }
```

The most crucial part of the logic was managing arrays of objects safely. The `Module` class below shows how I add and remove crew members by checking for null values to prevent crashes, and how I match the exact UUID to remove the correct person.

Listing 4: Module class (Array management)

```
1 import java.util.UUID;
2
3 public class Module {
4     private CrewMember[] crewMembers;
5     private int maxCapacity;
6
7     public Module(int maxCapacity) {
8         this.maxCapacity = maxCapacity;
9         this.crewMembers = new CrewMember[maxCapacity];
10    }
11
12    public void addCrewMember(CrewMember newMember) {
13        for (int i = 0; i < crewMembers.length; i++) {
14            if (crewMembers[i] == null) {
15                crewMembers[i] = newMember;
16                System.out.println(newMember.getName() + " added to "
                    module.);
17                return;
18            }
19        }
20        System.out.println("Module is full!");
21    }
22 }
```

```

23     public void removeCrewMember(UUID targetId) {
24         for (int i = 0; i < crewMembers.length; i++) {
25             if (crewMembers[i] != null && crewMembers[i].getId().equals
                (targetId)) {
26                 System.out.println("Removed_crew_member_from_module.");
27                 crewMembers[i] = null;
28                 return;
29             }
30         }
31         System.out.println("Crew_member_not_found_in_this_module.");
32     }
33 }

```

2.3 Tester Class

To verify the implementation of the new Space Station features, I created a `MasterTester` class. It checks the creation of officers and crew members, the generation of their unique UUIDs, and the removal system.

Listing 5: Space Station `MasterTester` class

```

1 import java.util.UUID;
2
3 public class MasterTester {
4
5     public static void main(String[] args) {
6
7         SpaceStation myStation = new SpaceStation(5, 3, 3);
8         System.out.println("---_Space_Station_Initialized_---");
9
10        Officer commander = new Officer("Donald_Tusk", "Commander", "
            Command", 30);
11        CrewMember engineer = new CrewMember("Stanislaw_Blaszczyk", 28,
            Role.ENGINEER, "Software_management", 850.5);
12
13        System.out.println("Officer_hired:_ " + commander.getName());
14        System.out.println("ID_Badge_printed:_ " + commander.getId());
15        System.out.println();
16
17        System.out.println("Crew_hired:_ " + engineer.getName());
18        System.out.println("ID_Badge_printed:_ " + engineer.getId());
19        System.out.println();
20
21        myStation.addOfficer(commander);
22        myStation.addCrewMember(engineer);
23        System.out.println("---_Personnel_added_to_the_Station's_main_
            database!_---");
24        System.out.println();
25
26        System.out.println("---_Testing_the_Removal_System_---");
27
28        UUID badgeToRemove = engineer.getId();
29
30        System.out.println("Attempting_to_remove_crew_member_with_ID:_ "
            + badgeToRemove);
31
32        myStation.removeCrewMember(badgeToRemove);

```

```
33 |     }
34 | }
```

2.4 Console Output

Example output produced by the `MasterTester` program:

```
--- Space Station Initialized ---
Officer hired: Donald Tusk
ID Badge printed: ee971b74-e814-4829-b551-08d1118424e6

Crew hired: Stanislaw Blaszczyk
ID Badge printed: de1fbf08-ca2b-42a4-8fda-2cb92c7718b8

--- Personnel added to the Station's main database! ---

--- Testing the Removal System ---
Attempting to remove crew member with ID: de1fbf08-ca2b-42a4-8fda-2
cb92c7718b8
Crew member removed from station array.
```

3 Conclusions

- I learned how to use encapsulation to protect class variables. Making fields `private` and using getters ensures the object's data cannot be changed illegally from outside.
- I practiced using static factory methods (`create()`). This is a very safe way to create objects because I can validate all the data before calling the constructor.
- I learned how to use the `UUID` class to generate unique identifiers, which makes it easy to find or remove specific objects (like crew members) in arrays.
- I successfully managed array loops, remembering to check if an index is `null` before trying to access the object's methods or checking its ID.